

AI-Augmented Backend Development Project

TITLE:

AI BlogNest API – AI-Powered Blog Content Management System

Project Description

AI BlogNest API is an AI-Augmented backend web application developed using MongoDB, Express.js, and Node.js, integrated with Artificial Intelligence capabilities using Google Gemini AI. The system is designed to simplify blog creation, content management, and AI-assisted writing. It provides secure REST APIs for user authentication, blog management, comments, categories, and content operations. The application uses JWT authentication and role-based access control to protect resources. AI capabilities such as AI-powered blog content generation and content summarization help users create high-quality, context-aware blog content efficiently. The system combines traditional backend development with modern AI technologies to provide a secure, scalable, and intelligent content management platform.

Scenario

Problem:

Blog creators often spend significant time planning, writing, organizing, and managing blog content. Managing multiple drafts, categories, tags, comments, and publishing activities manually can become difficult as the number of blog posts increases. Traditional content management systems also require users to create most of the content manually, which can make the overall blogging process time-consuming and less efficient.

solution:

AI BlogNest API addresses these problems by integrating Artificial Intelligence with a secure backend system. Google Gemini AI is used to assist users with AI-powered blog content generation and content summarization. The application provides REST APIs for

creating and managing blogs while JWT authentication and role-based access control protect the system. AI assistance reduces manual effort and helps users generate and process content more efficiently.

Impact:

AI BlogNest benefits content creators, authors, editors, administrators, and readers by providing a centralized platform for blog management and AI-assisted content creation. Authors can create and manage blog posts, editors can review and manage content, administrators can control users and system activities, and readers can access published blogs. The application improves the efficiency of content creation while maintaining security, organization, and scalability.

System Requirements

To ensure smooth development, deployment, and usage of the AI BlogNest API AI-Augmented Backend Application, certain system prerequisites must be met.

1. Software Requirements

Operating System

Windows 10/11

macOS

Linux

Node.js (v16 or above)

Provides the runtime environment required for backend development and execution.

npm (v8 or above)

Package manager used for installing and managing project dependencies.

Express.js

Backend framework used for developing REST APIs and handling server-side requests.

MongoDB

NoSQL database used to store:

User Information

Blog Posts

Categories

Comments

Analytics Data

Mongoose

ODM used for defining database schemas, validating data, and interacting with MongoDB.

Google Gemini AI

AI service used for:

AI-powered blog content generation

Content summarization

AI-assisted content creation

Postman / Thunder Client

API testing tools used to test and verify the REST API endpoints.

Visual Studio Code

Development environment used for writing, editing, and managing the AI BlogNest backend project.

Git and GitHub

Used for source-code version control and project management.

Environment Variables

Used to securely store sensitive configuration values such as:

MongoDB connection string

JWT secret

Gemini API key

2. HARDWARE REQUIREMENTS

Processor

Intel Core i5 (8th Generation or above) / AMD Ryzen 5 or equivalent.

RAM

Minimum 8 GB

Recommended 16 GB

Storage

Minimum 1 GB free project workspace.

Internet Connection

Required for package installation, database connectivity, and Google Gemini AI API communication.

Display

1366 × 768 or above.

3. PROJECT ARCHITECTURE

Technical Architecture

AI BlogNest follows a modular layered backend architecture using Node.js, Express.js, MongoDB, JWT authentication, and Google Gemini AI.

Backend Layer

Node.js

Express.js

REST APIs

JWT Authentication

Role-Based Authorization

Database Layer

MongoDB

Mongoose ODM

Users

Posts

Categories

Comments

Analytics Data

AI Layer

Google Gemini AI

AI Blog Generation

Content Summarization

Prompt-Based Content Generation

Security Layer

JWT Authentication

Protected Routes

Password Hashing

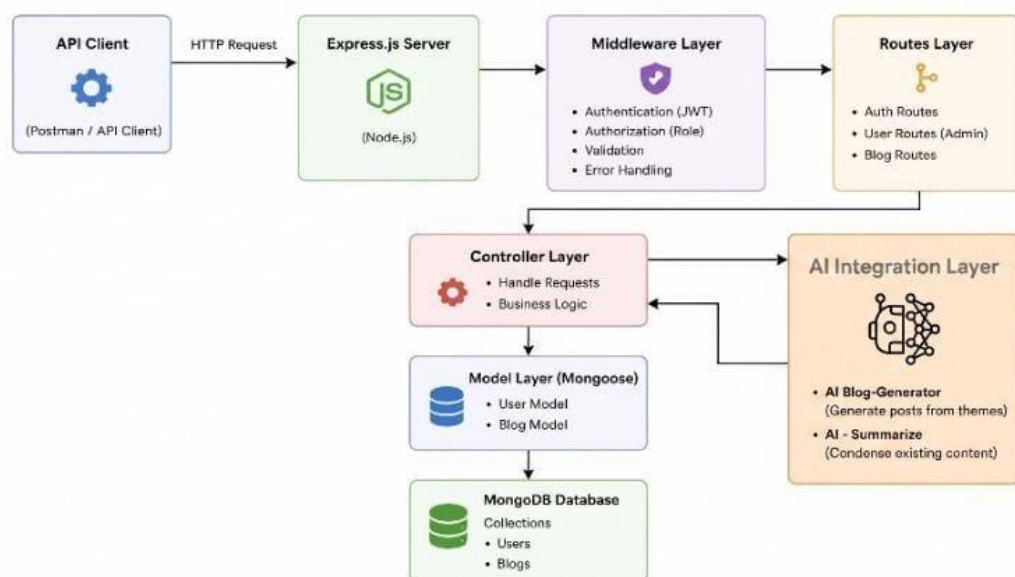
Input Validation

Input Sanitization

Rate Limiting

Role-Based Access Control

Architecture Diagram



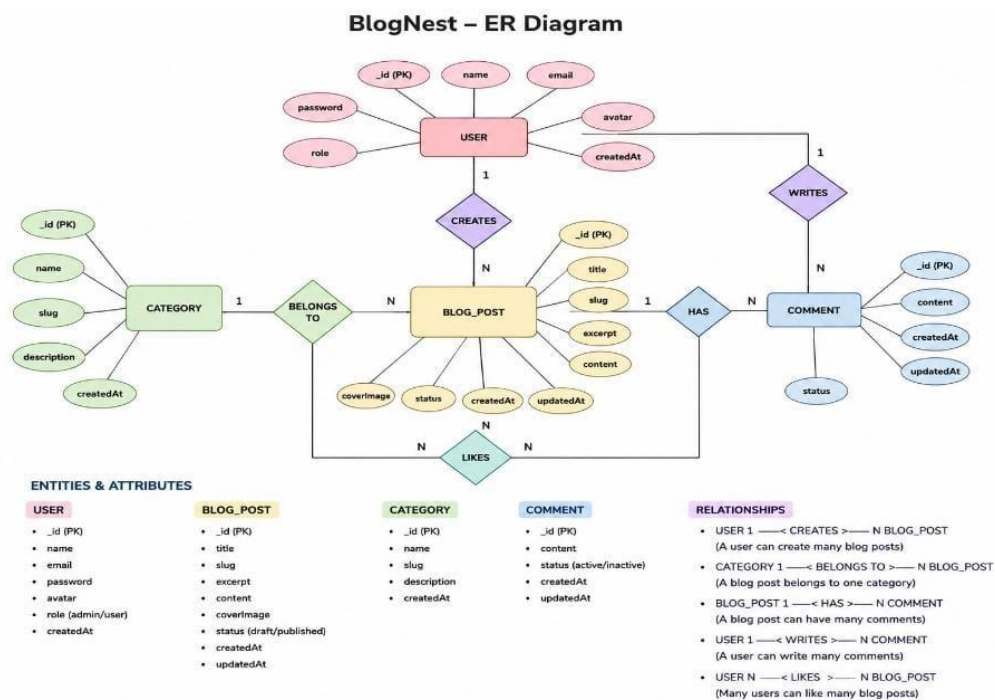
Categories

Caption:

Figure 1: AI BlogNest API Technical Architecture

4. ER DIAGRAM

The underlying document structure manages several collections bound by logical application references:



Users

_id

name

email

password

role

Posts

_id

userID

name

photo

message

likes

createdAt

updatedAt

Comments

_id

postId

userId

comment

status

createdAt

Categories

_id

name

description

Analytics

Stores information related to blog activity and performance where implemented.

ER Diagram Explanation

Users can create and manage blog posts based on their assigned role. A single user can create multiple posts, while a blog post can contain multiple comments. Comments are associated with users and posts. Categories and tags help organize blog content and improve content discoverability.

ER Diagram

Diagram:

USER

|

| 1 : Many

↓

POST

|

| 1 : Many



COMMENT

POST ----- CATEGORY

Caption:

Figure 2: AI BlogNest API Entity Relationship Diagram

5. FEATURES

Core Backend Features

User Registration

User Login

JWT Authentication

Protected Routes

Role-Based Access Control

Blog CRUD Operations

Draft Management

Publishing Status

Categories and Tags

Comment Management

Comment Moderation

Media Metadata Management

Input Validation

Input Sanitization

Rate Limiting

Structured Logging

AI Features

AI-Powered Blog Generation

Content Summarization

AI-Assisted Content Creation

Context-Aware Content Generation

The AI BlogNest reference specifically describes Gemini AI integration for AI-powered blog generation and content summarization. [?](#)

AI BlogNest API.docx

6. ROLES AND RESPONSIBILITIES

Admin

Manage users

Change user roles

Manage comments

Monitor system activities

Inspect logging information

Editor

Review draft posts

Approve posts

Publish posts

Manage categories

Author

Create blog posts

Manage their own posts

Prepare draft content

Reader

Read published blogs

Create comments

Access public content

AI BlogNest API.docx

7. USER FLOW

User Registration

User Login

JWT Authentication

Role Verification

Access Protected APIs

Create Blog

Store Blog in MongoDB

Generate AI Content if required

Review Blog

Publish Blog

Reader Views Published Blog

Reader Adds Comment

Registration

↓

Login

↓

JWT Authentication

↓

Role Verification

↓

Create / Manage Blog

↓

MongoDB

↓

AI Content Generation

↓

Review / Publish

↓

Reader

↓

Comments

Caption:

Figure 3: AI BlogNest API User Flow

8. MVC PATTERN

The AI BlogNest backend follows the MVC (Model-View-Controller) architecture pattern.

Model Layer

Responsible for:

User Schema

Post Schema

Comment Schema

Category Schema

Database Validation

Implemented using Mongoose.

Controller Layer

Responsible for:

Authentication Logic

Blog CRUD Operations

Comment Operations

AI API Integration

Content Generation

Content Summarization

Request Processing

View Layer – API Routes

Since AI BlogNest is a headless backend application, traditional frontend views are not used. API routes act as the interface between client requests and controllers.

Authentication Routes

Register

Login

Blog Routes

Create

Read

Update

Delete

Comment Routes

Create Comment

Manage Comments

AI Routes

Generate Blog

Summarize Content

Advantages of MVC

Separation of Concerns

Better Maintainability

Easy Scalability

Reusable Components

Easier Testing

The reference AI BlogNest document describes the same headless API routing approach for its MVC structure. [\[2\]](#)

AI BlogNest API.docx

Routes

↓

Controllers

↙ ↘

Models AI Service

↓ ↓

MongoDB Gemini AI

Caption:

Figure 4: MVC Architecture of AI BlogNest API

9. PROJECT SETUP AND CONFIGURATION

Project Structure

Create a project folder named:

AI-BlogNest

Inside the project:

Server

Open the Server folder in Visual Studio Code.

Server Setup

```
mkdir server
```

```
cd server
```

```
npm init -y
```

Create:

```
server.js
```

Create folders:

```
models/
```

```
controllers/
```

```
routes/
```

```
middlewares/
```

```
config/
```

```
services/
```

```
utils/
```

Install Backend Packages

Use the packages actually used by your project. The reference project uses Node.js, Express.js, MongoDB/Mongoose, authentication and AI integration. [?](#)

AI BlogNest API.docx

Example:

```
npm install express mongoose cors dotenv bcryptjs jsonwebtoken
```

For Gemini AI, use the Gemini package configured in your actual project.

For development:

```
npm install nodemon --save-dev
```

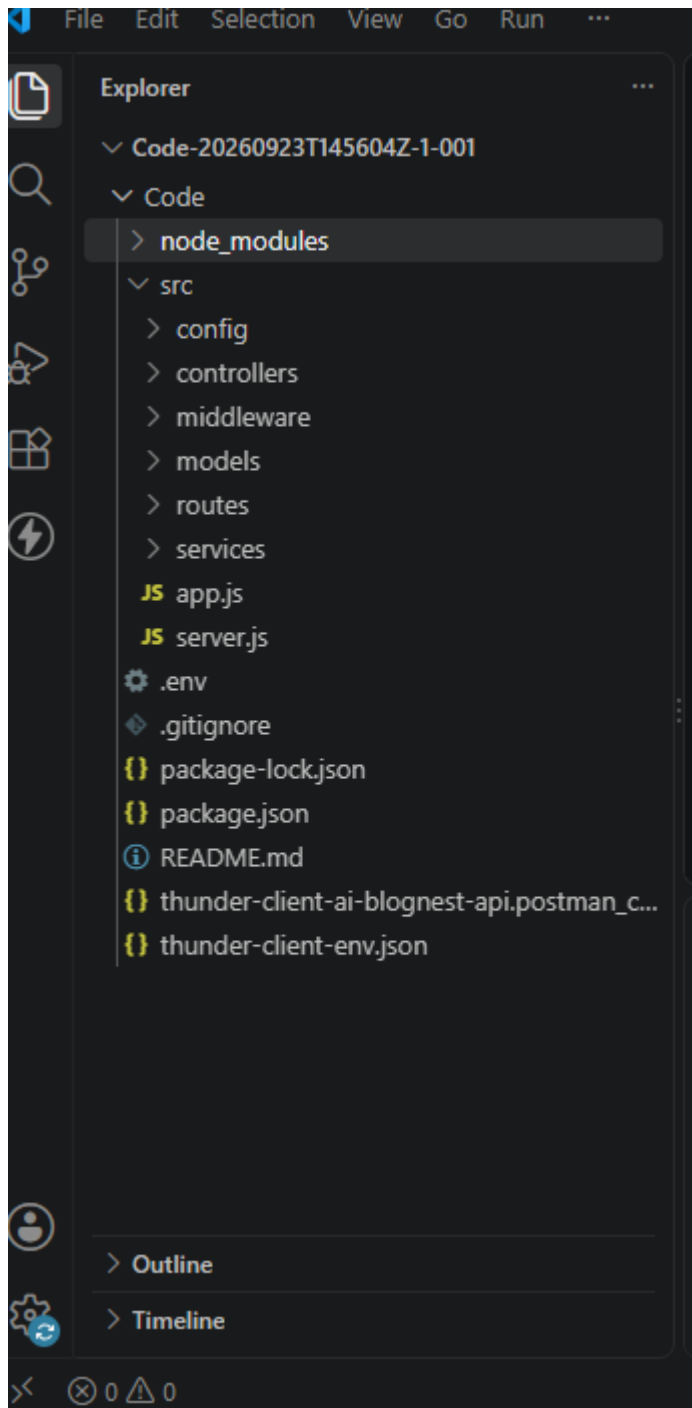


Figure 5: AI BlogNest API Project Structure in Visual Studio Code

10. BACKEND DEVELOPMENT

The backend provides REST APIs for authentication, blog management, comments, and AI-assisted content operations.

Authentication APIs

User Registration

User Login

JWT Authentication

Protected Routes

Blog APIs

Create Blog

Get All Blogs

Get Single Blog

Update Blog

Delete Blog

Comment APIs

Create Comment

Manage Comments

Comment Moderation

AI APIs

AI Blog Generation

Content Summarization

The AI BlogNest document specifically identifies AI generation and summarization endpoints. [?](#)

AI BlogNest API.docx

AI API Explanation

For each AI API, document:

Purpose:

Generate or summarize blog content using Google Gemini AI.

Request Structure:

The client sends a topic, prompt, or content through the API request.

Response Structure:

The backend returns the AI-generated blog content or summary as a JSON response.

Cost Considerations:

AI API usage may depend on the provider's usage limits and pricing.

Security Practices:

The Gemini API key must be stored in environment variables and must not be exposed in source code.

server.js

AI controller/service

routes

Caption:

Figure 6: AI BlogNest Backend API Implementation

11. DATABASE DEVELOPMENT

MongoDB is used as the primary database for AI BlogNest.

Database Collections

Users

Posts

Comments

Categories

Analytics

Database Relationships

Users can create multiple posts.

Posts can contain multiple comments.

Users can create comments on posts.

Mongoose is used for schema definition, validation, and database interaction.

Open MongoDB Compass and show:

Database

Collections

Documents

Caption:

Figure 7: AI BlogNest Database and Collections in MongoDB Compass

Open the Posts collection and show one blog document.

Caption:

Figure 8: Sample Blog Post Document in MongoDB

12. PROJECT EXECUTION

Step 1: Backend

Open the Server folder:

```
cd server
```

```
npm install
```

Step 2: Configure Environment Variables

Create:

```
.env
```

Example:

```
PORT=8000
```

```
MONGO_URI=your_mongodb_connection_string
```

```
JWT_SECRET=your_secret_key
```

```
GEMINI_API_KEY=your_gemini_api_key
```

Do not show the actual secret/API key in screenshots.

Step 3: Run Backend

Use your actual project command, for example:

```
npm run dev
```

or:

```
nodemon server.js
```

Backend URL:

```
http://localhost:8000
```

Use the port actually shown by your server.

Server running...

MongoDB connected...

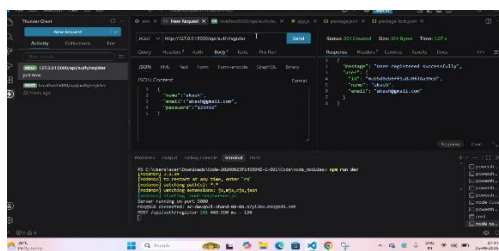
or whatever successful output your project actually displays.

Caption:

Figure 9: AI BlogNest API Server Running Successfully

13. OUTPUT SCREENSHOTS WITH EXPLANATION

API Testing

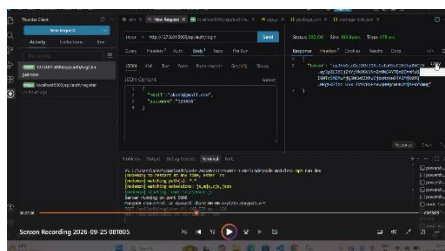


User Registration API

Show POST request + successful response.

Figure 10: User Registration API Testing

User Login API



Show login request and JWT response.

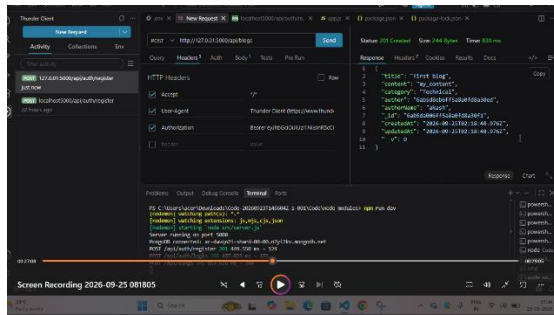
Caption:

Figure 11: User Login and JWT Authentication

Create Blog API

Show blog creation request and response.

Figure 12: Create Blog API Testing

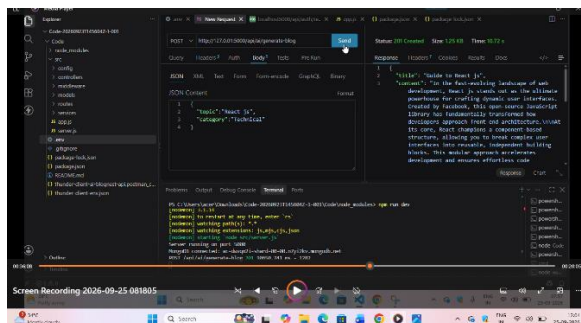


Get Blogs API

Show the list of blogs returned.

Caption:

Figure 13: Blog Retrieval API Testing

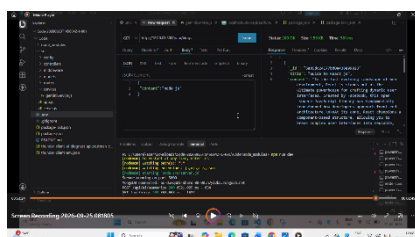


AI Blog Generation

Show the request/prompt and AI-generated blog response.

Caption:

Figure 14: AI-Powered Blog Generation



AI Summarization

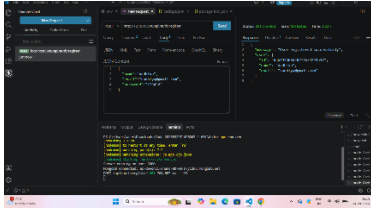
Show the content sent and generated summary.

Caption:

Figure 15: AI Content Summarization

PHOTO 16

MongoDB Output



Show stored blog data in MongoDB Compass.

Caption:

Figure 16: Blog Data Stored in MongoDB

14. DEMO VIDEO LINK

Demo Video:

<https://drive.google.com/file/d/1azakKjMU-dod1WJQs7bvbP1eoODnmHJi/view?usp=sharing>

15. SOURCE CODE DRIVE LINK

Source Code:

<https://drive.google.com/drive/folders/1S69VGxkcmkrsPhfJ6CE9af4Nf9WVnLsj?usp=sharing>

16. MANDATORY FUNCTIONALITIES

For your AI BlogNest API, list the functionalities that are actually implemented:

MVC Architecture

JWT Authentication

Role-Based Access Control

MongoDB Database

REST API Development

Blog CRUD Operations

Comment Management

AI API Integration

AI Blog Content Generation

AI Content Summarization

Prompt Engineering,Input Validation,Input Sanitization,Secure Password Hashing